

Geospatial Search with 4D, ORDA and 4D.Vector

By Karim Meghraoui, Technical Support Engineer, 4D Morocco.

Technical Note 26-09

Table of Contents

Table of Contents	2
Abstract.....	3
Introduction	3
General Overview of the Application	3
The Application's Geographic Data	5
GPS Coordinate Representation	6
Importing Geographic Data into the 4D Database	7
Distance Calculation Between Two Locations	10
Input Validation	10
Unit Conversion.....	11
The Haversine formula.....	11
Role in the Application.....	12
Geospatial search with ORDA.....	12
Parameter Validation.....	14
Using Formula() in an ORDA query	14
Sorting results by proximity.....	15
Building the result collection.....	15
Map Visualization of Results	16
Technologies involved	17
Building the map	18
Displaying an Interactive Map in a 4D Web Area.....	18
Sending dynamic data from 4D to the web area	19
Updating the display on the web area side	20
Displaying a single position	20
Displaying two positions and a route	21
Conclusion	22

Abstract

Modern applications increasingly use geolocation to provide services tailored to users' geographic positions. Today, it has become common to search for a nearby restaurant, locate a bank branch, find a hotel in a city, or calculate the distance between two locations. All these features rely on the use of geospatial data.

A geospatial datum is information associated with a position on the Earth's surface. This position is generally represented by two numerical values: latitude and longitude. Latitude indicates a point's position relative to the equator, while longitude indicates its position relative to the Greenwich meridian. The combination of these two values makes it possible to precisely identify any location in the world.

Using these coordinates, an application can perform various operations, such as measuring the distance between two points, searching for objects located within a geographic area, or displaying information on an interactive map. These operations are collectively referred to as geospatial search.

There are many application domains such as:

- Tourism platforms that enable users to discover points of interest near a city
- Delivery applications that search for the nearest driver to a customer
- Transportation companies that track the position of their vehicles in real time
- Real estate platforms that offer properties within a given geographic area
- In all these examples, location is a key element of the search process.

Introduction

This technical note presents the implementation of a geospatial search feature built with 4D, ORDA, and the 4D.Vector type. The application uses a database containing Moroccan cities as well as various tourist sites in Morocco. Starting from a city selected by the user, it automatically searches for all sites located within a specified radius, calculates their distance, and displays the results as both a list and an interactive map.

The purpose of this technical note is to explain the different steps involved in implementing this feature, from storing coordinates to displaying them on a map. Each main method of the project will be presented to demonstrate how the different 4D components can be combined to build a simple and efficient geospatial solution.

General Overview of the Application

The application is based on a relatively simple principle: each city and each tourist site has a GPS location stored in the database. These coordinates are then used to calculate distances between different locations.

When a user performs a search, they first select a city from a list. This city then becomes the reference point for all calculations. The user subsequently chooses a maximum distance, expressed in kilometers. This distance defines the search radius.

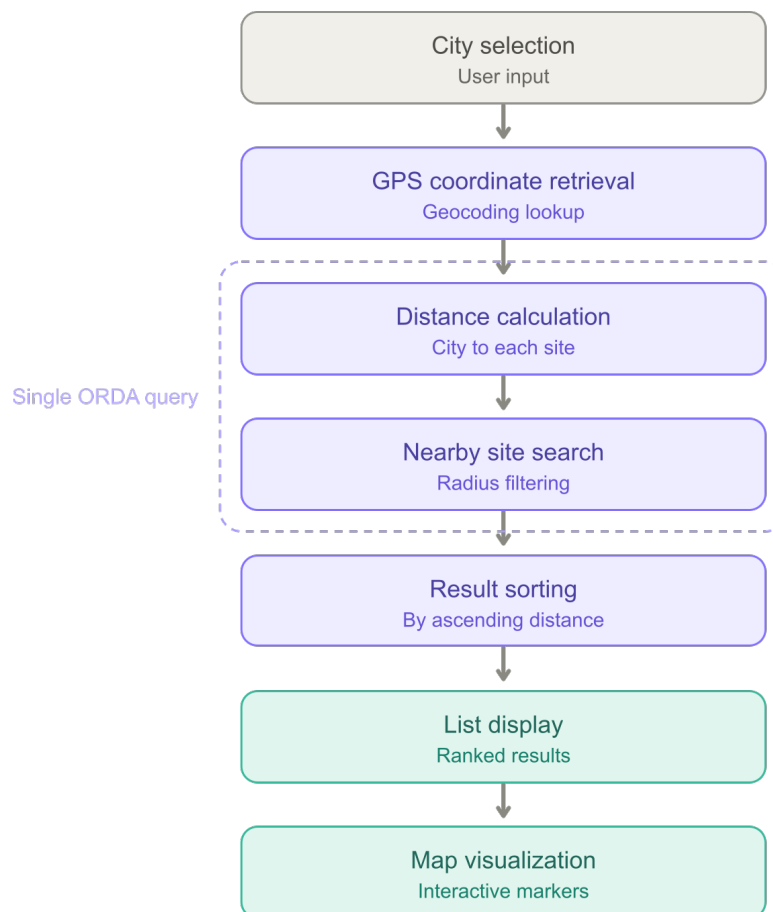
The application retrieves the GPS coordinates of the selected city and compares them with those of every tourist site stored in the database. For each site, the distance is calculated. If this distance is less than or equal to the selected radius, the site is included in the results. All of this is executed as a single ORDA query within the 4D database engine.

Let us consider a simple example. Suppose the user selects the city of Marrakesh and specifies a search radius of 50 kilometers. The application will examine all available tourist sites. If a site is located 18 kilometers from Marrakesh, it will be added to the results list. Conversely, a site located 120 kilometers away will not be included.

The resulting sites are then automatically sorted from the nearest to the farthest. This ranking enables users to immediately identify the most accessible locations.

In addition to the results list, the application generates an interactive map displaying the selected locations. This graphical representation makes it easier to understand the results by showing the positions of the sites directly on a map.

The overall workflow of the application can be summarized as follows:



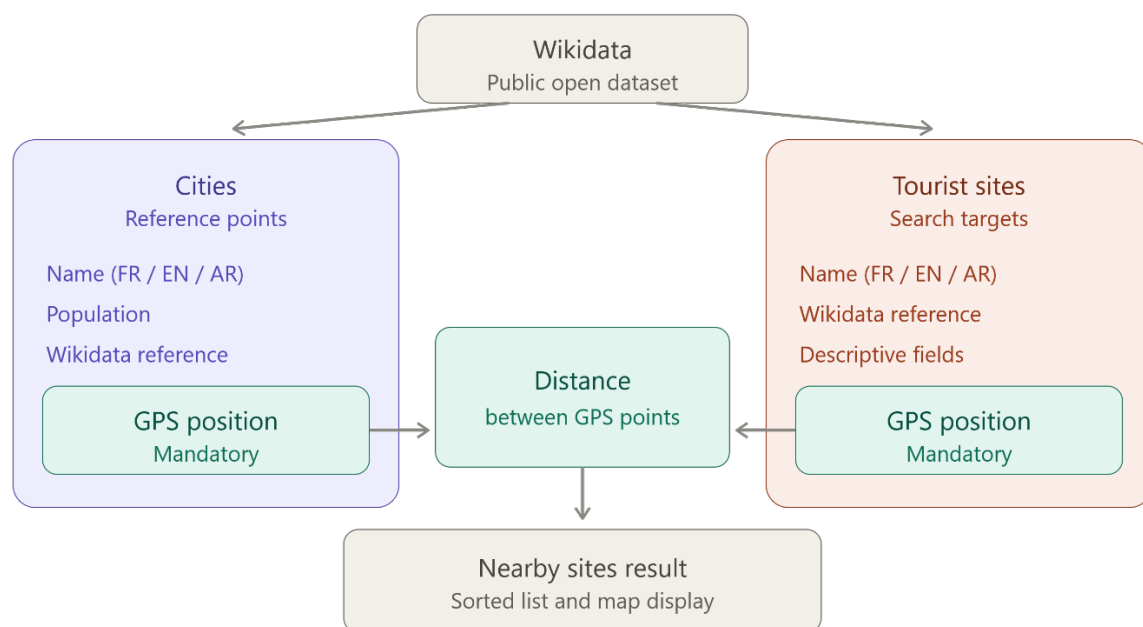
This search principle is now widely used in mapping applications, tourism platforms, and mobility services.

The Application's Geographic Data

In this application, the information is organized around two types of entities: cities and tourist sites. Wikidata was used to gather a small dataset of moroccan cities and touristic sites. Cities serve as reference points for searches. Each city contains several descriptive attributes, such as its name in French, English, and Arabic, its population, and a Wikidata reference linking to a page containing all known information about that city.

Tourist sites represent the locations that users wish to search for. Each site also contains several descriptive attributes, including its name in different languages and its Wikidata reference.

However, the most important piece of information for both entities is their geographic position. Without it, it would be impossible to calculate distances or display locations on a map.



Unlike a traditional approach that would use two separate numeric fields (latitude and longitude) for the coordinate, this application uses the 4D.Vector type. This allows both coordinates to be grouped into a single variable.

For example:

[31.6295,-7.9811]

The first value corresponds to the latitude. The second corresponds to the longitude.

Using 4D.Vector offers several advantages.

First, the code becomes easier to read because coordinates are handled as a single object. The various methods no longer need to receive two separate parameters for latitude and longitude. Instead, they work directly with a vector containing both values.

In addition, 4D.Vector instances are immutable, which reduces the risk of accidental modifications that could overwrite either the latitude or the longitude.

For a comparison of the two approaches (separate latitude/longitude and a single 4D.Vector), here are the method declarations for a 4D method that may receive two coordinates to compute a distance:

```
#DECLARE ($latitude1 : Real; $longitude1 : Real; $latitude2 : Real; $longitude2 : Real) ->
$distance : Real
```

```
#DECLARE($coords1 : 4D.Vector; $coords2 : 4D.Vector) -> $distance : Real
```

This approach is also more robust against development errors. When latitude and longitude are stored in two separate variables, they can be inadvertently swapped. This risk is significantly reduced when coordinates are handled as a 4D.Vector instance.

Finally, this approach makes future application enhancements easier. All methods use the same data type, reducing the likelihood of errors and improving the overall consistency of the project.

GPS Coordinate Representation

Geographic coordinates are stored in the database using decimal degrees, which is the format used by most modern GPS systems.

For example, the coordinates of Marrakech can be represented as:

31.6295

-7.9811

These values are particularly well suited for mathematical calculations, but they are sometimes less intuitive for users to read.

In certain contexts, especially in mapping or tourism applications, it is preferable to use a more traditional representation known as the DMS format (Degrees, Minutes, Seconds).

A coordinate such as:

31.6295

becomes:

31°37'46"N

This format is closer to the one commonly found on maps and in some navigation systems.

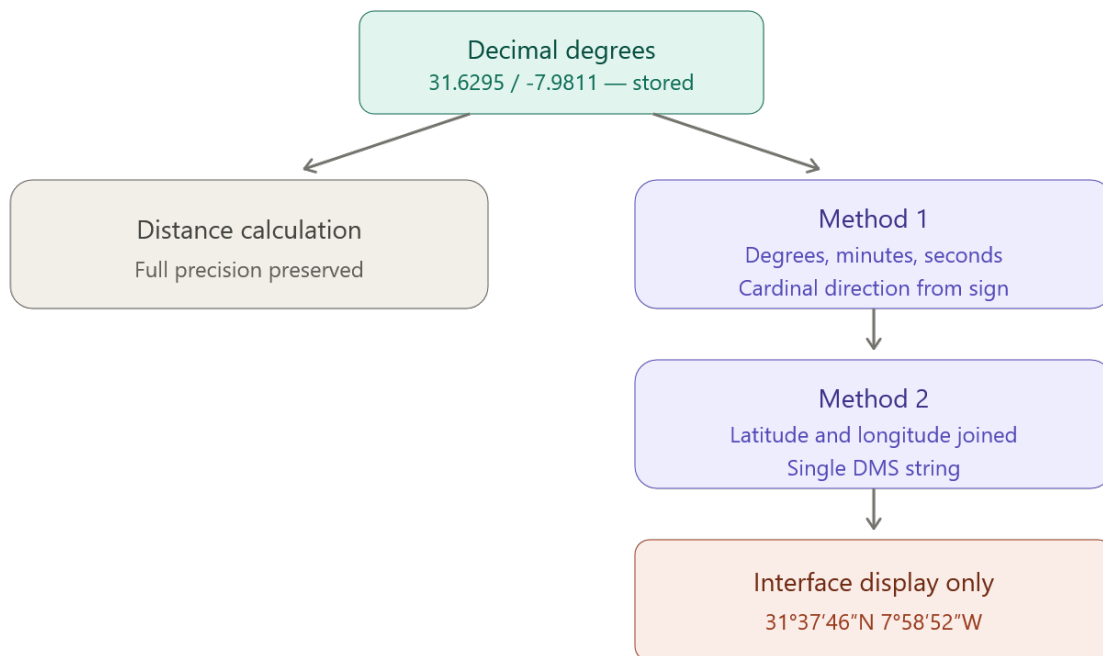
The application therefore includes two methods dedicated to this conversion.

The first method converts an individual latitude or longitude into degrees, minutes, and seconds. It also determines the corresponding cardinal direction (North, South, East, or West) based on the sign of the value.

The second method combines the latitude and longitude to produce a complete representation such as:

31°37'46"N 7°58'52"W

This conversion is not used for distance calculations. Distance calculations continue to be performed using decimal coordinates in order to preserve their full precision. The DMS format is used solely to improve the presentation of geographic information in the user interface.



Coordinate formats: computation path and display path.

Importing Geographic Data into the 4D Database

Before a geospatial search can be performed, the application must have a dataset containing cities and tourist sites. This information is provided through two JSON files located in the application's RESOURCES folder:

- morocco-cities.json, which contains information about cities;
- touristic-sites.json, which contains information about tourist sites.

The JSON format is a simple and lightweight format that is widely used for data exchange and is fully supported by 4D. It is also easy for users to read and can be directly processed by 4D using the JSON Parse() command.

The data import process is performed only once, during database initialization. Before starting, the method verifies that the tables do not already contain data.

```
If ((ds.City.getCount()#0) || (ds.TouristicSite.getCount()#0))
```

```
// Data tables `City` and/or `TouristicSite` already contain data.  
  
return  
  
End if
```

This verification is important. If the tables already contained records, running the import again could create duplicate entries. The method therefore stops the process and informs the user that the data already exists.

Once this verification has been completed, the two JSON files are read and converted into collections of objects.

```
$citiesData:=JSON Parse(File("/RESOURCES/morocco-cities.json").getText())  
$touristicSitesData:=JSON Parse(File("/RESOURCES/touristic-sites.json").getText())
```

The File().getText() command reads the content of the file, while JSON Parse() transforms the text into a collection of objects that can be manipulated directly in the 4D language.

Each object in the collection corresponds to a city or a tourist site.

The method then iterates through each collection to create the corresponding records in the database.

For cities, a new ORDA entity is created using the new() function.

```
$cityEntity:=ds.City.new()
```

This instruction creates a new CityEntity object, which is then populated with information read from the JSON file.

The different properties are copied into the entity.

```
$cityEntity.nameFR:=$cityData.nameFr  
$cityEntity.nameEN:=$cityData.nameEn  
$cityEntity.population:=Num($cityData.population)
```

The same process is applied to other attributes such as names in different languages and the Wikidata reference.

GPS coordinates, however, require special processing.

In the JSON file, coordinates are stored as an array containing two values.

For example:

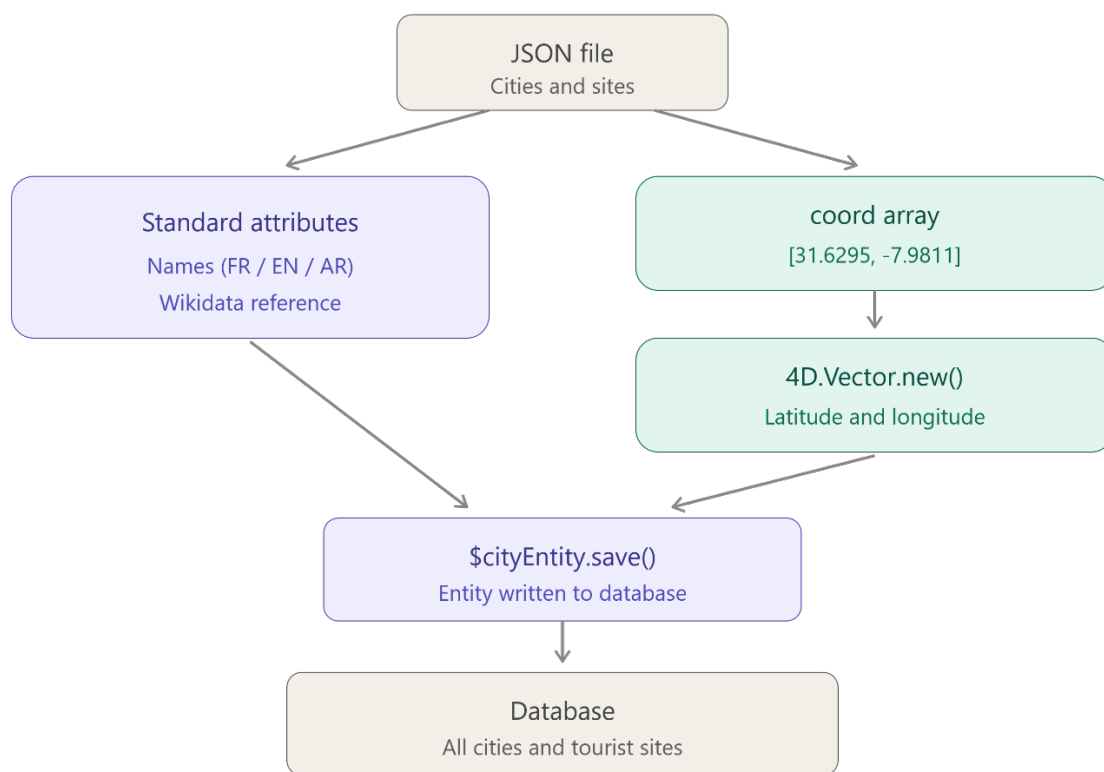
```
[31.6295,-7.9811]
```

To make them easier to manipulate within the application, they are converted into a **4D.Vector** object.

```
$cityEntity.coordinates:=4D.Vector.new($cityData.coord)
```

This instruction automatically transforms the JSON array into a vector containing the latitude and longitude values.

Using the 4D.Vector type offers various advantages: besides simplifying the implementation of methods or routines that manipulate coordinates, it is also fully supported as a storable type in the 4D database engine and known to the 4D compiler which can apply syntax checking to detect errors and misuses.



Importing JSON data into ORDA entities

Once all properties have been assigned, the entity is saved into the database.

```
$cityEntity.save()
```

The same process is applied to tourist sites.

At the end of this step, the database contains all the cities and tourist sites required for geospatial searches.

This method is a classic example of importing data from JSON files into ORDA entities.

Distance Calculation Between Two Locations

Once coordinates are stored in the database, distances between locations can be computed. This operation is handled by the `geoDistance()` method, which has a very simple signature: it receives two GPS coordinates, and returns the distance between them, expressed in kilometers.

Input Validation

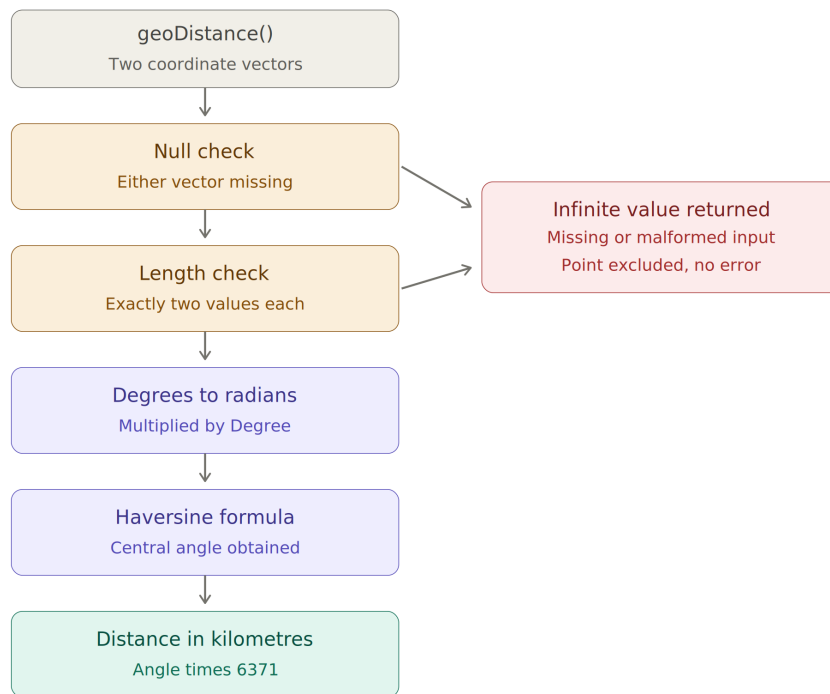
Two guard clauses run before any computation. The first one rejects missing coordinates:

```
If (($coordsA=NULL) || ($coordsB=NULL))  
    return Num("INF")  
End if
```

The second one confirms that each vector holds exactly two components:

```
If (($coordsA.length#2) || ($coordsB.length#2))  
    return Num("INF")  
End if
```

Returning an infinite value rather than raising an error is a deliberate design choice: an infinite distance excludes the point from proximity searches automatically, with no special-case handling anywhere else in the application.



Control flow of the `geoDistance()` method

Unit Conversion

Coordinates are stored in decimal degrees, whereas the trigonometric functions provided by 4D operate in radians. Each component is therefore converted before use:

```
$latA:=$coordsA[0]*Degree
$lonA:=$coordsA[1]*Degree
```

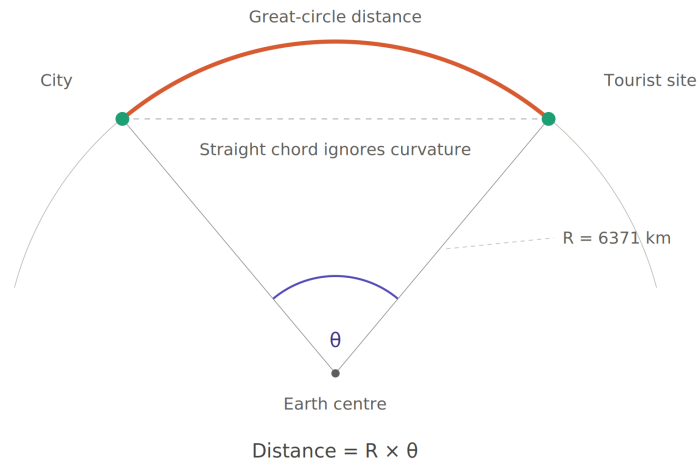
The same conversion applies to the second pair of coordinates.

The Haversine formula

The method then applies the Haversine formula, the standard approach in geolocation systems for measuring the shortest path between two points on the surface of the Earth. Unlike a planar distance, it accounts for the curvature of the globe.

The formula yields the central angle separating the two positions. Multiplying that angle by the mean Earth radius, gives the distance:

```
return $earthRadius*$CentralAngle
```

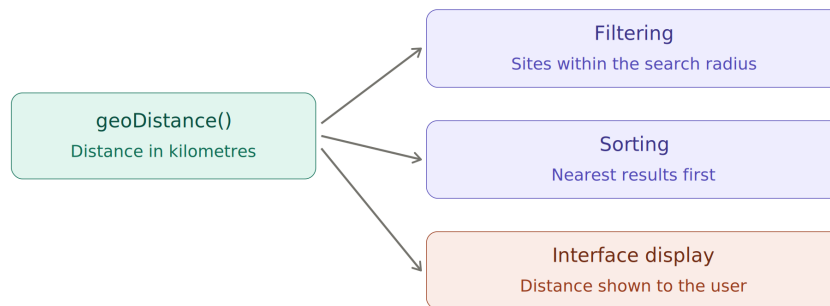


Haversine principle: the central angle scaled by the Earth radius.

Modelling the Earth as a sphere of constant radius keeps the deviation from the WGS84 ellipsoid below half a percent, which is negligible for proximity searches.

Role in the Application

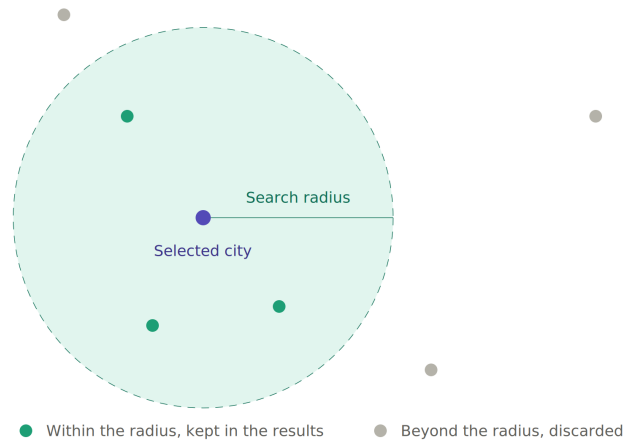
This method is the core of the application: every geospatial feature depends on its output. The computed distance filters tourist sites against the search radius, orders the results from nearest to furthest, and feeds the distance shown to the user in the interface.



Downstream uses of computed distance

Geospatial search with ORDA

With the `geoDistance()` method available, the remaining task is to use it to perform an actual search operation. For example, if a user wants to find every touristic site located within a given radius (for example 30 km) around a city (let's say "Casablanca"), only the touristic sites closer than 30 kilometers should be returned.



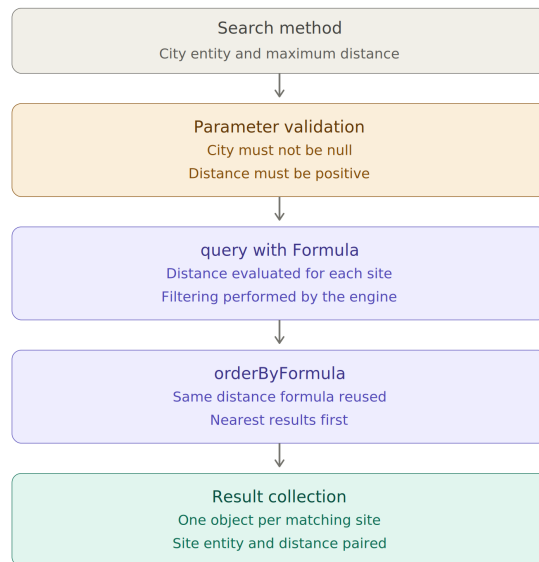
Sites are kept or discarded according to their distance from the selected city

4D answers this use case with ORDA (Object Relational Data Access), the data access framework provided by 4D. ORDA exposes stored data as entities and entity selections rather than as SQL result sets: each record becomes an entity carrying its own properties and methods. Tourist sites are reached through the datastore, where `ds.TouristicSite` represents the entity selection covering every site in the database.

The search method takes two parameters, the reference city and the maximum distance, and returns a collection:

```
#DECLARE($city : cs.CityEntity; $maxDistance : Real)->$sitesWithDistance : Collection
```

The first parameter is the selected city entity; the second is the search radius, expressed in kilometres. The search then proceeds in four stages.



Sequence of the search method, from parameters to result collection

Parameter Validation

Before any search runs, the method asserts that the supplied parameters are usable:

```
ASSERT($city#Null)
ASSERT($maxDistance>=0)
```

The first assertion guarantees that a city has been provided, the second that the requested distance is positive. Together they prevent a search from running on invalid input.

Using Formula() in an ORDA query

The heart of the search is a single ORDA query:

```
$sites:=ds.TouristicSite.query(
  Formula(
    geoDistance(This.coordinates,$city.coordinates)<=$maxDistance
  )
)
```

The formula is evaluated in an efficient way by the database engine once for every tourist site in the table. For each site it compares two values:

- the coordinates of the site being examined, exposed as `This.coordinates`;

- the coordinates of the selected city, passed in as `$city.coordinates`.

A site is retained when its distance to the city is less than or equal to `$maxDistance` — in other words, when the site falls inside the search area.

This approach is efficient because no records are loaded into memory to be measured one by one in application code: the filtering is delegated to the ORDA engine.

Sorting results by proximity

A geographic search is expected to present results from the nearest location to the farthest. The same distance formula serves as the sort criterion:

```
.orderByFormula(
  Formula(
    geoDistance(This.coordinates;$city.coordinates)
  )
)
```

The distance is computed for every matching site and drives the ordering, so that a search around Marrakech returns, for instance:

Site	Distance
Bahia Palace	0.8 km
Majorelle Garden	2.4 km
Oukaïmeden	48.5 km

The closest locations are therefore immediately identifiable at the top of the list.

Building the result collection

Once the search is completed, the method assembles a collection pairing each matching site with its distance. The collection starts empty:

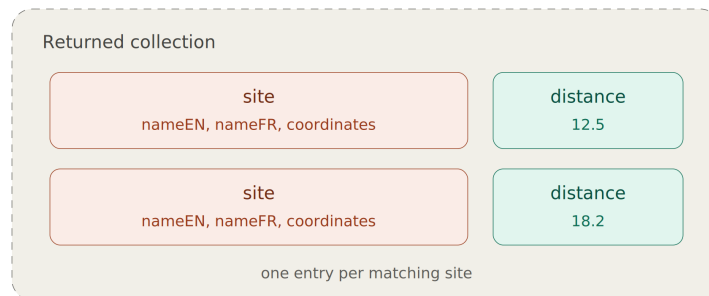
```
$sitesWithDistance:=[]
```

Each result is then iterated over, its distance computed, and an object pushed into the collection:

```
For each ($site;$sites)
  $distance:=geoDistance($site.coordinates;$city.coordinates)
  $sitesWithDistance.push({
```

```
site : $site;  
distance : $distance  
})  
End for each
```

The returned collection therefore holds one object per site, each carrying the site entity and its distance:



Structure of the returned collection

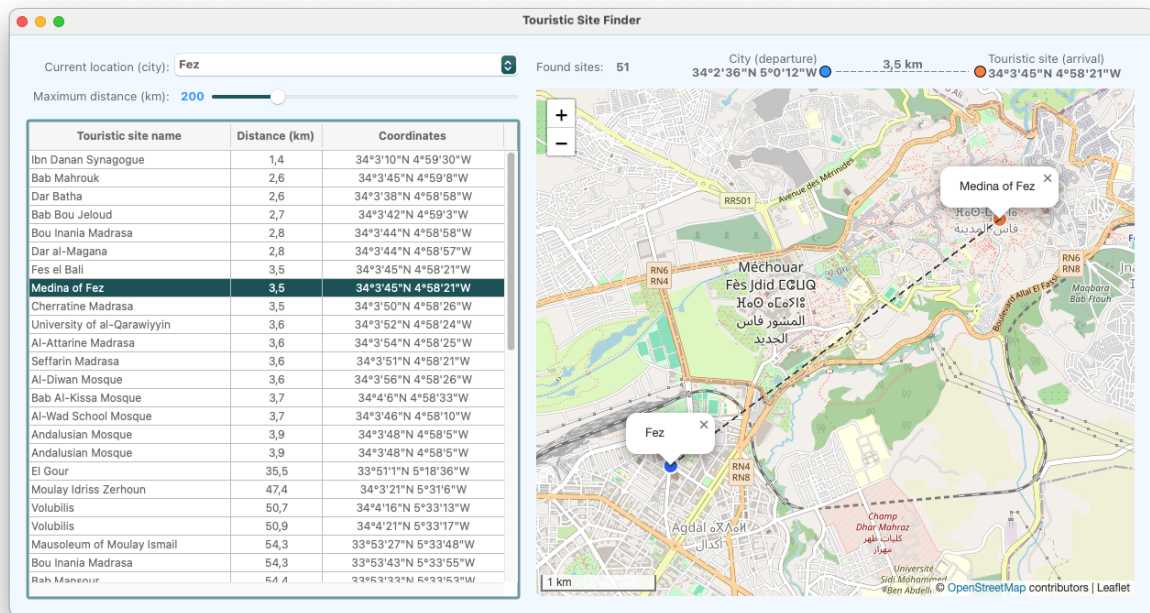
This organization simplifies the interface layer, since the results list can read every value it needs from a single object:

- ✓ the name of the site
- ✓ its associated descriptive information
- ✓ the computed distance

ORDA thus provides a concise and efficient way to perform geospatial searches while keeping the code object-oriented and readable.

Map Visualization of Results

After performing the geospatial search and obtaining the list of tourist sites near a city, the application provides a graphical representation of the results in the form of an interactive map.

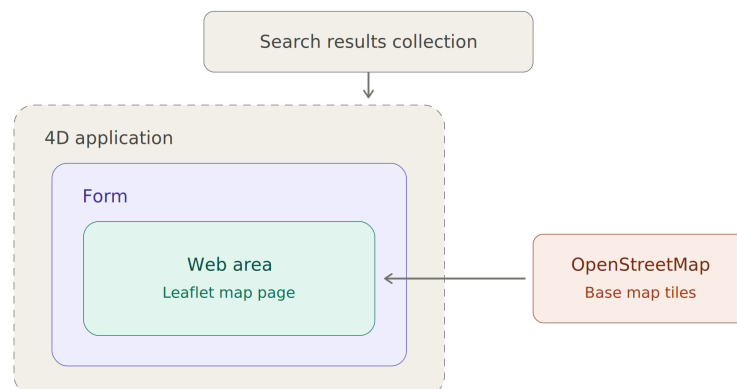


Technologies involved

Three technologies combine to deliver the feature:

- OpenStreetMap supplies the map data used as the base layer
- 4D web area embeds the map inside a form
- Leaflet renders the map and handles the interactivity

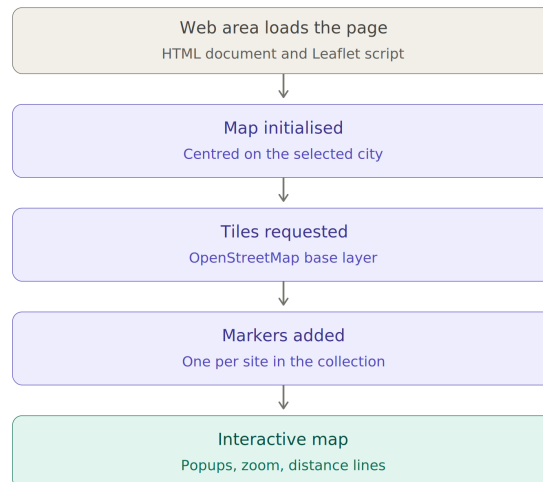
Leaflet is a lightweight JavaScript library for building interactive maps in a web page. It covers every need of this feature: displaying a map, placing markers, drawing lines between points, opening information popups, and controlling the zoom level.



Components involved in displaying the map

Building the map

The web area loads an HTML document that references the Leaflet library. The map is then centered on the selected city, the base layer is requested from the OpenStreetMap tile servers, and one marker is created for each entry of the collection returned by the search, where every object pairs a site with its distance.



Sequence followed to build the map

The resulting map is fully interactive: a click on a marker opens a popup carrying the name of the site and its distance to the city, lines can be drawn between two positions, and the zoom level remains under the control of the user.

Displaying an Interactive Map in a 4D Web Area

The map page, map.html, sits in the Resources folder of the project. It loads the Leaflet library and exposes a single JavaScript entry point, `updateMap()`, responsible for refreshing the display.

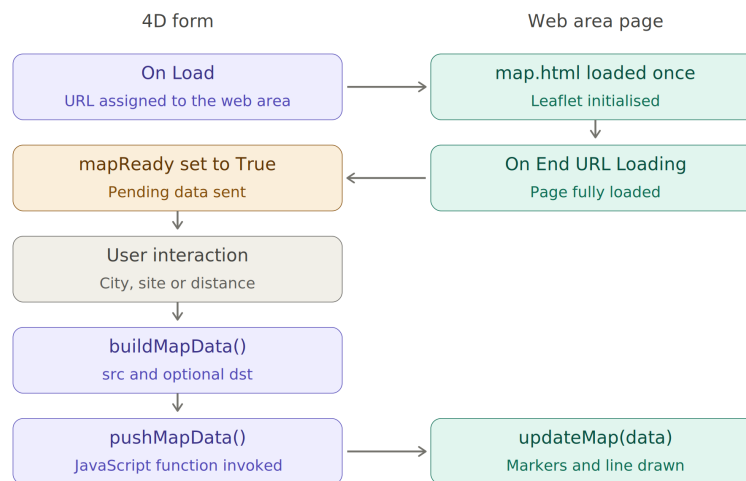
The web area loads that page from a 4D Web Server running locally:

```
#DECLARE->$url : Text
startMapServer()
$url:="http://127.0.0.1:8044/map.html"
```

The URL is assigned to the web area on the On Load event of the form:

```
mapViewURL:=mapPageURL()
```

The page is therefore loaded once and only once. Later changes to the city, the site or the search distance never reload it; only the displayed data changes.



Exchange between the 4D form and the page hosted in the web area

Sending dynamic data from 4D to the web area

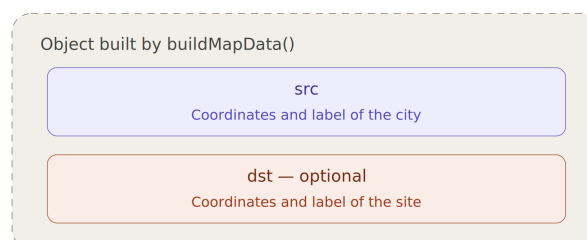
Only the coordinates and the labels of the locations vary from one display to the next. The buildMapData() method groups them into a single object:

```

$data:={}
$data.src:={coords: $city.coordinates.toCollection(); label: $city.nameEN}
If ($site#Null)
    $data.dst:={coords: $site.coordinates.toCollection(); label: $site.nameEN}
End if
  
```

The resulting object holds at most two properties:

- src : the starting position, a city, with its coordinates and its label;
- dst : the destination, a tourist site, present only when a site is selected.



Structure of the object sent to the page

The object is sent to the static page by the `pushMapData()` method, which relies on the WA EXECUTE JAVASCRIPT FUNCTION command:

```
WA EXECUTE JAVASCRIPT FUNCTION(*; "webMapView"; "updateMap"; *; $data)
```

The command executes the JavaScript `updateMap()` function on the page and passes the 4D object as its parameter; the conversion into a JavaScript object is automatic.

Loading a page in a web area is asynchronous, so `updateMap()` can only be called once the page is fully loaded. The On End URL Loading event signals that moment, and a `mapReady` flag records the state. While the page is not ready the data is prepared but withheld; as soon as loading completes, any pending data is sent:

```
// On End URL Loading event
mapReady:=True
If (mapData#Null)
    pushMapData(mapData)
End if
```

Every subsequent interaction — choosing a city, selecting a site, changing the distance — rebuilds the data and sends it only if the page is ready:

```
mapData:=buildMapData(citySelected; nearbySiteSelected.site)
If (mapReady)
    pushMapData(mapData)
End if
```

Updating the display on the web area side

The `updateMap()` function receives the object sent by 4D and updates the map, distinguishing two cases.

Displaying a single position

When only the starting data is supplied, a single blue marker is drawn and the map is centred on that position:

```
var srcMarker = L.circleMarker(data.src.coords, {
    radius: 8,
    weight: 2,
```

```
        color: 'fff',
        fillColor: '#1e66ff',
        fillOpacity: 1
    }).addTo(overlay);
```

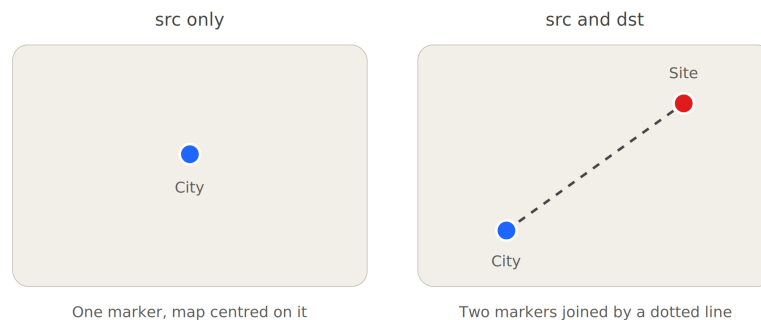
Displaying two positions and a route

When destination data is present as well, the same blue marker is kept for the city, a second marker in red is added for the site, and a dotted line joins the two:

```
var dstMarker = L.circleMarker(data.dst.coords, {
    radius: 8,
    weight: 2,
    color: 'fff',
    fillColor: '#ff8040',
    fillOpacity: 1
}).addTo(overlay);
var line = L.polyline([data.src.coords, data.dst.coords], {
    color: '#444',
    weight: 2,
    dashArray: '6,6'
}).addTo(overlay);
```

The zoom level is then adjusted so that both points remain visible at the same time:

```
map.fitBounds(line.getBounds(), { padding: [100, 100] });
```



The two display cases handled by `updateMap()`

The relationship between the city and the tourist site is therefore immediately visible, with no manual panning or zooming after each change of selection.

Conclusion

This technical note presents the implementation of a complete geospatial search feature in an application developed with 4D.

The project shows how several features can be combined to build a modern location-based solution.

Using **4D.Vector** makes it possible to represent GPS coordinates simply as an object containing a latitude and a longitude. This approach makes geographic data easier to handle across all methods of the application.

Thanks to **ORDA**, searches can be performed directly on database data using a custom formula based on distance calculation. The `geoDistance()` method, which uses the Haversine formula, provides a reliable distance between two positions on Earth.

The dynamic generation of maps with **Leaflet** and **OpenStreetMap** provides a visual representation of the results and improves the user experience. Users can therefore consult sites near a city both as a list and as a graphical display.

Although the example presented concerns Moroccan cities and tourist sites, this architecture can be adapted to many domains: searching for nearby customers, locating branches, managing deliveries, tracking vehicles, or finding services within a geographic area.

This project therefore demonstrates that a 4D application can effectively integrate modern geospatial features without requiring complex infrastructure. Combining 4D, ORDA, 4D.Vector, and Web mapping tools makes it possible to create a simple, scalable solution suited to the current needs of business applications.